

# Web Programming CoSc3312\_

June 15, 2025

Compiled by: Natan Asrat

[https://www.linkedin.com/in/natan\\_asrat](https://www.linkedin.com/in/natan_asrat)

# Web Programming

- World Wide Web (WWW) or the Web: a network of computers all over the world
  - Web pages are files stored on computers called Web servers
  - Computers reading the Web pages are called Web clients
- Web hosting: the place where all the files of your website live
  - Web host (web hosting service provider): a business providing the technologies and services needed for the website or webpage to be viewed in the Internet
  - Domain name registration: reserving a name on the Internet for a certain period

# Web Programming

- HTML (HyperText Markup Language) – since 1990 – the most basic building block of the Web.
  - not a programming language
  - it is a markup language that tells web browsers how to structure the web page
  - defines the meaning and structure of web content
  - uses CSS to describe appearance/presentation and/or JavaScript for functionality/ behavior

# Web Programming

- HTML
  - HTML tag: used for creating an element; delimits the start and end of elements in the markup.



# Web Programming

- HTML – HTML tag:

- HTML Attributes: special words used to control the behavior of an element
- Meta elements: tags for providing structured metadata about a web page

```
<meta charset="utf-8">
```

- HTML Links (Hyperlinks): allow us to link documents to other documents or resources, links to specific parts of documents

```
<a href="https://www.google.com">Google</a>.
```

- Text Formatting tags: Headings (predefined formats for presentation), Paragraph

# Web Programming

- HTML – HTML tag:
  - Image inserting tag:
    - <img>: void element requiring at least one attribute to be useful, (src)
    - image map: a list of coordinates relating to a specific image (clickable links)

```
<map name="primary">  
  <area shape="circle" coords="75,75,75" href="left.html">  
  <area shape="circle" coords="275,75,75" href="right.html">  
</map>  

```

# Web Programming

- HTML – HTML tag:
  - HTML Table: for structuring tabular data

```
<table>
  <caption align="center" valign="bottom">table 1.0</caption>
  <tr>
    <th>Column 1</th>
    <th>Column 2</th>
  </tr>
  <tr>
    <td>Cell 1</td>
    <td>Cell 2</td>
  </tr>
  <tr>
    <td>Cell 3</td>
    <td>Cell 4</td>
  </tr>
</table>
```

# Web Programming

- HTML – HTML tag:
  - Ordered (ol) and Unordered List (ul)

```
<ul type="square">
  <li>book</li>
  <li>marker</li>
  <li>chalk</li>
</ul>
<ol>
  <li>...</li>
  <li>...</li>...
</ol>
: type="number type" {1, i, I, a, A}
```

```
<dl>
  <dt>...</dt>
  <dd>...</dd>
  <dt>...</dt>
  <dd>...</dd>
</dl>
```



# Web Programming

- HTML – HTML tag:
  - HTML Frames: present documents in multiple views, (independent windows or sub-windows)
    - Frame Set: specifies the layout of views in the main user agent window
      - contain a NOFRAMES element to provide alternate content for user agents that do not support frames or are configured not to display frames.
      - attributes: Cols, Rows, Border
    - Internal Frame: define the particular area within an HTML file where another HTML web page can be displayed

```
<!DOCTYPE html>
<html>
<head>
  <title>Frame tag</title>
</head>
<frameset cols="25%,50%,25%">
  <frame src="frame1.html" >
  <frame src="frame2.html">
  <frame src="frame3.html">
</frameset>
</html>
```

# Web Programming

- HTML - HTML tag:
  - HTML Form and Form Controls: interactive controls for submitting information.
    - attributes: action (backend script), method, target (\_blank, \_self, \_parent), enctype (application/x-www-form-urlencoded - common, mutlipart/form-data - binary data)

# Web Programming

- HTML – HTML tag:
  - Inserting Multimedia: to add(embed) images or videos
    - <embed>: embed a video, audio and gif files to website, but the browser doesn't know media type ahead of time

```
<embed src = "video" width = "100%" height = "60" >  
    <noembed>  
        <img src = "image" alt = "Alternative Media" >  
    </noembed>  
</embed>
```

- <video>: allow to embed a video

```
<video src="video location" controls>  
    <p>fallback text </p>  
</video>
```

- <audio>: embeds audio

<audio controls>

<source src="audio source" type="type of audio like audio/mp3">

<source src="audio source" type="type of audio like ">

<p> if the browser doesn't support this feature it will fallback to this

# Web Programming

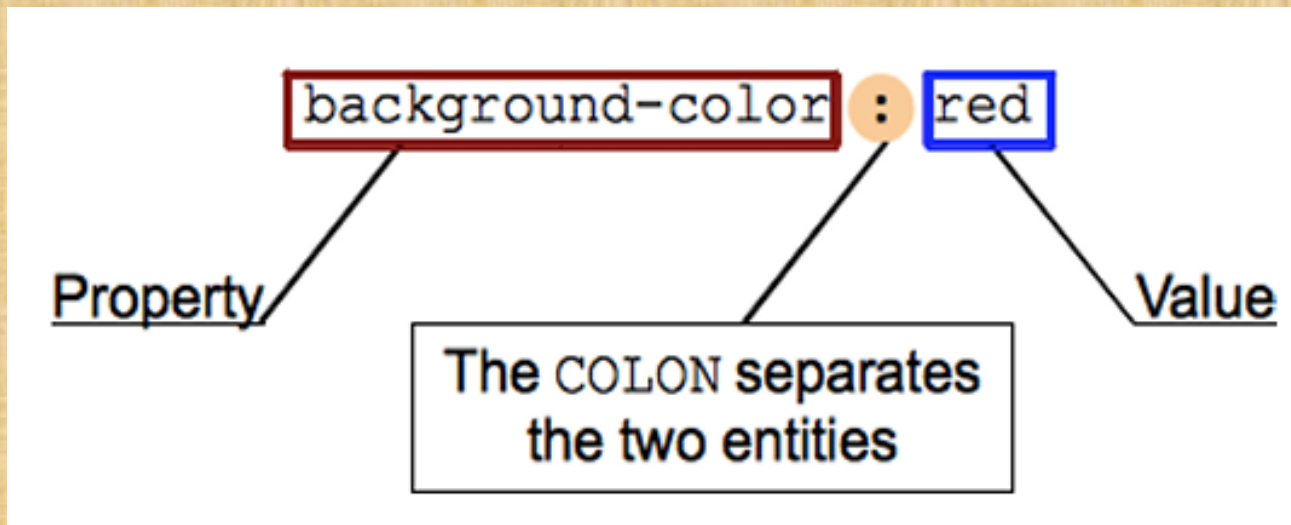
- HTML - HTML tag:
  - HTML Graphics: visual representations; HTML Canvas (2D or 3D scene - animation, game), HTML SVG (<svg> to embed an SVG fragment)

```
<svg width="600"  
      height="150">  
  <rect width="600"  
        height="150"  
        fill="green"  
        stroke="black"  
        stroke-width="6" />  
</svg>
```



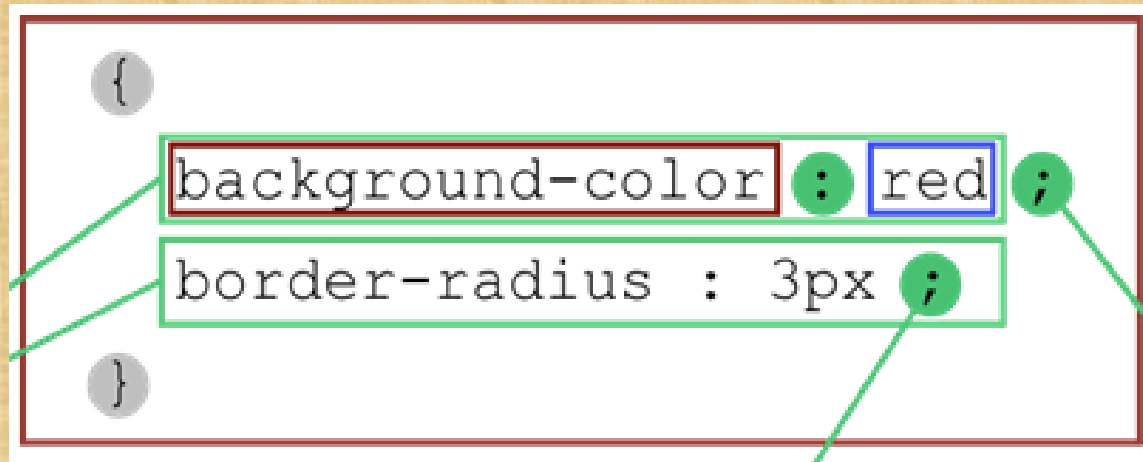
# Web Programming

- Cascading Style Sheets (CSS)
  - CSS: used to selectively style HTML elements
  - building block of CSS syntax: property (identifier) & value (how the feature must be handled by the engine)
  - CSS Declarations: sets CSS properties to specific values



# Web Programming

- Cascading Style Sheets (CSS)
  - Declaration Block: groups declarations delimited by a brace `{`



# Web Programming

- Cascading Style Sheets (CSS)
  - Attaching CSS with HTML:
    - Inline stylesheet: specified for an individual element via style attribute

```
<p style="color: red; background:green;">Inline style</p>
```

# Web Programming

- Cascading Style Sheets (CSS)
  - Attaching CSS with HTML:
    - Embedded stylesheet: used when a single document has a unique style

```
<head>
  <style>
    H1 {
      color:yellow;
      Text-align:left;
    }
    P{
      margin-left:20px;
    }
  </style>
</head>
```



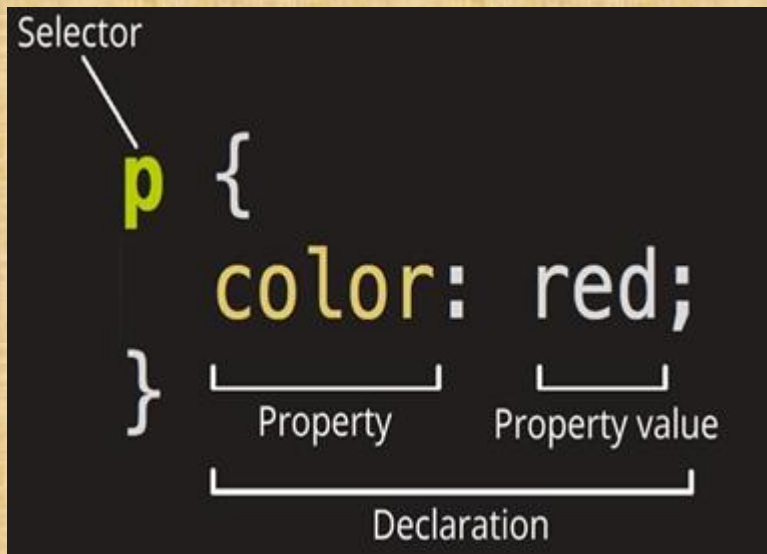
# Web Programming

- Cascading Style Sheets (CSS)
  - Attaching CSS with HTML:
    - External style sheets: used when the style is applied to many pages; <link> tag inside the head section

```
<link rel="stylesheet" href="style.css">
```

# Web Programming

- Cascading Style Sheets (CSS)
  - Style Sheet Rules: selector group (HTML element name) and an associated declarations block



# Web Programming

- Cascading Style Sheets (CSS)
  - Ruleset: whole structure
  - Selectors: Type selector (by tag i.e. img), Class selector (multiple elements with specified class in their class attribute), Id selector (element with specified ID in id attribute), Attribute Selector (elements having the specified attribute), Pseudo-class selector (depends on state - i.e. hover)

# Web Programming

- Cascading Style Sheets (CSS)
  - Style Inheritance: uses the document tree for inheritance
    - Specificity: how the browser decides which rule applies if multiple rules have different selectors, but could still apply to the same element
    - Cascade: when two rules apply that have equal specificity, the one that comes last in the CSS is the one that will be used

```
p {  
    color: brown;  
}  
p {  
    color: green;  
}
```



# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - Foreground and Background Properties:
    - background shorthand CSS property sets all background style properties at once (color, image, origin and size, or repeat method) – can be image
    - color property sets foreground
  - Font and Text Properties:
    - font CSS shorthand property sets all the different properties (font-family, font-size, font-stretch, font-style, font-variant, font-weight)
    - Text Properties: text-transform(none, uppercase, lowercase, capitalize), text-decoration (none, underline, overline, line-through), text-align(left, right, center, justify)

# Web Programming

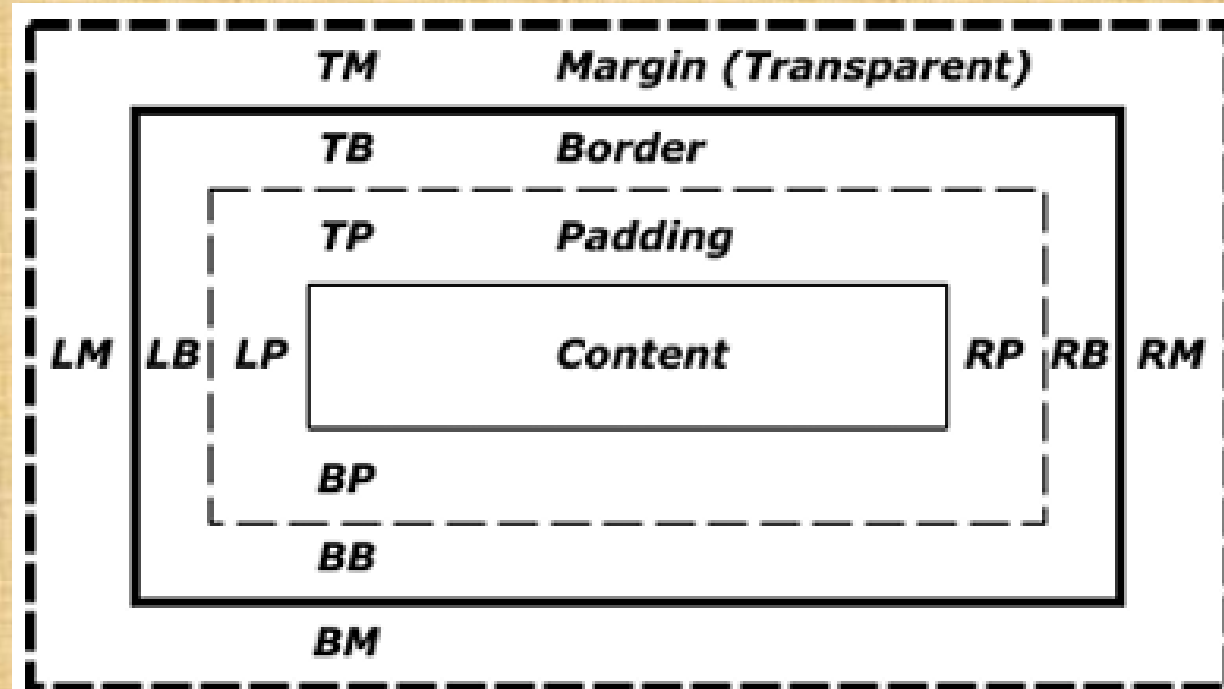
- Cascading Style Sheets (CSS) - Style Properties
  - List specific styles: `list-style-type`(type of bullets), `list-style-position`(start, inside, outside), `list-style-image`(custom image for bullet)

# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - CSS Box Model: everything in CSS has a box around it (block boxes and inline boxes)
    - Parts of a box
      - Content box: where your content is displayed, sized using width and height.
      - Padding box: padding sits around the content as white space
      - Border box: wraps the content and any padding

# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - CSS Box Model: everything in CSS has a box around it (block boxes and inline boxes)
    - Parts of a box
      - Margin box: outermost layer





# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - CSS Box Model:
    - Display CSS property: sets whether an element is treated as a block or inline element
      - block, inline, inline-block, flex, inline-flex, grid, inline-grid, etc

# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - Table Styling Properties: `table-layout(columns)`, `border-collapse(prevent spacing between borders)`
  - Layout and Positioning Properties: where elements are positioned relative to their default position in normal layout flow, the other elements around them, their parent container, and the main viewport/window

# Web Programming

- Cascading Style Sheets (CSS) - Style Properties
  - Layout and Positioning Properties - Main layouts:
    - normal flow,
    - flexbox(one-dimensional layout - main axis & cross axis),
    - grid(divides page into regions - grid-template-columns # of cols, grid-template-rows # of rows),
    - floats, table layout, multiple-column layout

# Web Programming

- Cascading Style Sheets (CSS) – Style Properties
  - Layout and Positioning Properties – Positioning:
    - Static Positioning (`position: static;`): default, normal position in the document flow
    - Relative Positioning (`position: relative;`): once the positioned element has taken its place in the normal flow, you can then modify its final position, including making it overlap other elements on the page; top, bottom, left, and right are used
    - Absolute Positioning (`position: absolute;`): no longer exists in the normal document flow; isolated UI that doesn't interfere with the layout of other elements
      - we have to set its container to position relative, or else it uses the documents body as relative



# Web Programming

- Cascading Style Sheets (CSS)
  - CSS Measuring Units
    - absolute units: cm, mm, in, pt, px
    - relative units: em (parent's font-size), rem (root element's font-size), vw (1% viewport width), vh (1% viewport height)

# Web Programming

- Client-Side Scripting (JavaScript)
  - JavaScript: a lightweight, interpreted or compiled programming language.
    - single threaded, dynamic, supporting object oriented, functional programming styles.
    - we can use JavaScript in coordination with Application programming interfaces (APIs)

# Web Programming

- Client-Side Scripting (JavaScript)
  - Internal JavaScript: one way of including JavaScript with script tag in our HTML document.

```
<script>  
  console.log("Hello world")  
</script>
```

- We can also include JavaScript as an external file.
  - defer attribute (Boolean): indicates to a browser that the script is meant to be executed after the document has been parsed, but before firing DOMContentLoaded. (has no effect if src attribute is not present)

```
<script src="script.js" defer></script>
```

# Web Programming

- Client-Side Scripting (JavaScript)
  - JavaScript Input Output: `var name = prompt("Enter your name: "); alert("Hello " + name);`
    - `console.log` (in the browser console), `alert` (popup), `document.write` (deletes html and writes on it)
  - JavaScript Data types: Number (integers and floats), String (single quote, double quote, back tick - to embed variables and expressions into a string), Boolean, Objects (key-value pair), Arrays (regular objects; store indexed values)
  - Variable declaration: `let` (block level scope), `var` (hoisted, function level scope), `const`



# Web Programming

- Client-Side Scripting (JavaScript)
  - Variable declaration: `let` (block level scope), `var` (hoisted, function level scope), `const`
  - Data Type Conversion: `String(value)` - converts to string, `Number(value)` - converts to number, `Boolean(value)`
  - Arithmetic operators: `+`, `-`, `*` (multiplication), `/` (division), `%` (remainder), `**` (exponent)
  - Logical Operators: `&&` (logical AND), `||` (logical OR), `!` (logical NOT)

# Web Programming

- Client-Side Scripting (JavaScript)
  - Control Structures (Conditional and Looping Statements):
    - if-else, ternary operators, loops (while, do-while, for)

```
let age = 20;

if (age < 18) {
    alert( 'Under...' );
} else if (age > 18) {
    alert( 'Over' );
} else {
    alert( '18!' );
}

while (condition) {
    // "loop body"
}

do {
    // "loop body"
} while (condition);

for (begin; condition; step) {
    // ... loop body ...
}

let result = condition ? value1 : value2;
```

# Web Programming

- Client-Side Scripting (JavaScript)
  - Array: store ordered collections; syntax - using new or [listing elements]
    - Methods: Push (append element), Pop (takes an element from the end), Shift (Extracts the first element of the array and returns it), Unshift (add element to the beginning of the array)
  - Functions: building blocks of a program; syntax - function declaration, arrow expression

```
function showCSMessage() {  
    alert( 'Hello CS Students!' );  
}  
  
    param => expression  
    (param1, paramN) => expression
```

# Web Programming

- Client-Side Scripting (JavaScript)
  - JavaScript DOM (Document object Model): data representation of the objects in the structure and content of the web
    - Accessing HTML elements in JavaScript:
      - `getElementById(id)`, `getElementsByClassName(class)`, `querySelectorAll(name)`, `element = document.querySelector(selectors)`
    - CSS in JavaScript: `element.style.backgroundColor = "red"`
    - Events: a signal that something has happened;
      - `element.addEventListener(event, handler, [options]);`
        - options: `once` (removes listener), `capture` (bubbling), `passive` (won't call `preventDefault`)
      - `element.addEventListener("click", handler, [options])`



# Web Programming

- Client-Side Scripting (JavaScript)
  - Handling Exception: `try {...} catch {...}`
  - Form Processing: intercept submit event,  
`form.addEventListener("submit", event ⇒ {...})`

# Web Programming

- Client-Side Scripting (JavaScript)
  - JavaScript BOM (Browser Object Model): to interact with the browser

# Web Programming

- Client-Side Scripting (JavaScript) - BOM
  - JavaScript Window: supported by all browsers, all variables, functions and js objects are members of window
    - `window.document.getElementById("header")`, `window.innerHeight`
    - `window.open()`: open a new window, `window.close()`: close the current window
    - `window.moveTo()`: move current window, `window.resizeTo()`: resize current window

# Web Programming

- Client-Side Scripting (JavaScript) – BOM
  - Window location: represents the location (URL)
    - window.location.**href**: URL of the current page
    - window.location.**hostname**: domain name of the web host
    - window.location.**pathname**: path and filename of the current page
    - window.location.**protocol**: web protocol used (http: or https:)
    - window.location.**assign()**: loads a new document



# Web Programming

- Client-Side Scripting (JavaScript)
  - JavaScript Cookies: data, stored in small text files, on your computer to remember information about the user

# Web Programming

- Client-Side Scripting (JavaScript) - Cookies:
  - saved in name-value pairs
  - when a browser requests a page, cookies of that page are added to the request
  - `document.cookie = "username=John Doe"`
  - can add an expiry date (in UTC time). default - cookie is deleted when browser is closed
    - `document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC";`
  - with a path parameter, you can tell the browser what path the cookie belongs to
    - `document.cookie = "username=John Doe; path=/"`

# Web Programming

- Server-Side Scripting (PHP)
  - PHP: a server-side scripting language; can be embedded in HTML or used as a standalone binary

# Web Programming

- Server-Side Scripting (PHP)
  - stands for Hypertext Preprocessor; originally named Personal Home Page Tools,
  - used to generate dynamic output (potentially different each time requested)
  - mostly about connecting Web sites to back-end servers, such as databases.
  - two-way communication: Server to client (assemble pages from back end), Client to server (process entered information)
  - by default PHP documents end with the extension .php



# Web Programming

- Server-Side Scripting (PHP)
  - Syntax

## Short-open (SGML-style) tags

Short or short-open tags look like this:

```
<?php  
//php script  
?>
```

```
<?
```

```
//php script
```

```
?>
```

# Web Programming

- Server-Side Scripting (PHP)
  - Predefined and User Variables: a variable does not need to be declared before adding a value to it
    - `$txt = "Hello World!"; $number = 10;`
    - `$GLOBALS`: reference to every variable in global scope of script.
    - `$_SERVER`: array containing information such as headers, paths, and script locations.
      - entries in this array are created by the web server.
    - `$_GET`: associative array of variables passed via the HTTP GET method.
    - `$_POST`: associative array of variables passed via the HTTP POST method.

# Web Programming

- Server-Side Scripting (PHP)
  - PHP Output Statements: for printing to output (browser window); echo and print - can be used with or without parentheses
    - echo "Hey user!"; echo("Welcome back!")
    - multiple arguments by comma: echo "Welcome to ", "our exam!";
      - only with echo and without parenthesis

# Web Programming

- Server-Side Scripting (PHP)
  - Data Types and Variables:
    - PHP is a very loosely typed language: no need to declare variables
    - PHP always converts variables to the type required by their context when accessed
  - Arithmetic and Logical Operators: to perform operations on the variables



# Web Programming

- Server-Side Scripting (PHP)
  - Conditional Statements: if - elseif -else, switch, ternary

```
<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

```
switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    case label3:
        code to be executed if n=label3;
        break;
    ...
    default:
        code to be executed if n is different
}
```

(condition) ? if TRUE execute this : otherwise execute this;

# Web Programming

- Server-Side Scripting (PHP)
  - Loop Statements: while, do-while, for, foreach

```
<?php
    for($y = 0; $y <= 10; $y++){
        echo "The value of Y is: $x <br>";
    }
?>
```

```
while (if the condition is true) {
    // code is executed
}
```

```
<?php
    $y = 1;

    do {
        echo "The value of Y is: $y <br>";
        $y++;
    } while ($y <= 5);

?>
```

```
$colors = array("red", "green", "blue", "yellow");
```

```
foreach ($colors as $x) {
    echo "$x <br>";
}
```

# Web Programming

- Server-Side Scripting (PHP)
  - Arrays in PHP: ordered map,
    - map can be treated as an array, list (vector), hash table, dictionary, collection, stack, queue, etc..
    - array syntax: using the array() construct, with [], with [index]

# Web Programming

- Server-Side Scripting (PHP) – Arrays

```
<?php
$array1 = array("Item 1", "Item 2", "Item 3", "Item 4");
```

```
$array2 = array('item1' => "Item 1",
                'item2' => "Item 2",
                'item3' => "Item 3",
                'item4' => "Item 4");
```

```
<?php
    $sample[0] = "sample 1";
    $sample[1] = "sample 2";
    $sample[2] = "sample 3";
    $sample[3] = "sample 4";
```

```
?>
    <?php
        $sample[] = "sample 1";
        $sample [] = "sample 2";
        $sample [] = "sample 3";
        $sample [] = "sample 4";
```

```
        print_r($sample);
    ?>
```



# Web Programming

- Server-Side Scripting (PHP)
  - PHP Functions: `function info($name,$age,$salary) {  
 echo ... }; info('Natan', 21, 0);`
  - Form Processing using PHP: `isset` (whether the variable in a form has value or not)

# Web Programming

- Server-Side Scripting (PHP) – Form Processing

```
<?php
if (isset($_POST['submit']))
{
    if ((!isset($_POST['firstname'])))

        {
            $error = "*" . "Please fill all the required fields";
        }
    else
    {
        $firstname = $_POST['firstname'];
    }
}
?>
```

# Web Programming

- Server-Side Scripting (PHP)
  - PHP File Upload: initially files are uploaded into a temporary directory and then relocated to a target destination
    - Configure The "php.ini" File to allow file uploads
    - form must use method='POST', and enctype='multipart/form-data'
    - extract file from \$\_FILES: \$\_FILES["fileToUpload"]

# Web Programming

- Server-Side Scripting (PHP) – Cookies and Session:
  - PHP session variable: to store information about, or change settings for a user session.
    - holds information about one single user, available to all pages in one application
  - cookie is often used to identify a user:  
`setcookie(name, value, expire, path, domain);`
    - `setcookie("user", "Alex Porter", time()+3600);`



# Web Programming

- Server-Side Scripting (PHP) – Cookies and Session

```
<?php
session_start();
if(isset($_SESSION['views']))
$_SESSION['views']=$_SESSION['views']+ 1;
else

$_SESSION['views']=1;
echo "Views=". $_SESSION['views'];
```

## Destroying a Session

```
<?php
    unset($_SESSION['views']);
?>
```

completely destroy the session

```
<?php
session_destroy();
?>
```

# Web Programming

- Server-Side Scripting (PHP) – Database Programming

- [https://www.w3schools.com/php/php\\_mysql\\_connect.asp](https://www.w3schools.com/php/php_mysql_connect.asp)
- Creating Database Connection:  
`$link=mysql_connect($hostname, $user, $password)`
- Selecting the database:  
`$db=mysql_select_db("dbname", $link);`
- Sending Query: `$result = mysql_query("query", $link);`

# Web Programming

- Server-Side Scripting (PHP) – Database Programming
  - Processing Query:
    - `while($row=mysql_fetch_row($result)){ echo "Name: " .  
    $row[1] }`
    - `while($row=mysql_fetch_object($result)){ echo "Name: " .  
    $row->name ; }`
    - `while($row=mysql_fetch_object($result)){ echo "Name: " .  
    $row {'name'} }`
  - Closing the connection: `mysql_close($link)`

# Web Programming

- Server-Side Scripting (PHP) – Input-Output:
  - `fopen & fclose`: `$file=fopen("hello.txt","r");`  
`fclose($file);`
    - modes: **r** (read only, starts at beginning of file), **r+** (read/write, starts at beginning of file), **w** (write only, open/create and clear file), **w+** (write only, open/create and clear file)
  - `readfile()`: reads a file and writes it to the output buffer
    - `echo readfile("webdictionary.txt")`



# Web Programming

- Server-Side Scripting (PHP) – Date and Time:
  - DateTime class handles dates themselves;
    - `$date = new DateTime('2025-06-10 14:00:00');`
    - `$date = new DateTime();` – now
  - DateTimeZone class handles time zones;
    - `$addisAbabaTimezone = new DateTimeZone('Africa/Addis_Ababa');`
    - `$date = new DateTime('2017-06-10 14:00:00', $addisAbabaTimezone);`
    - `$date = new DateTime('now', $addisAbabaTimezone);` – now

# Web Programming

- Server-Side Scripting (PHP) – Date and Time:
  - DateTimeInterval class handles spans of time between two DateTime instances;
    - \$startPeriod = new DateTime('2025-01-01');
    - \$intervalPeriod = new DateInterval('P1W'); – Every 1 week
    - \$endPeriod = new DateTime('2025-02-10');
  - DatePeriod: a set of dates and times, recurring at fixed intervals, between a start and an end date
    - \$period = new DatePeriod(\$startPeriod, \$intervalPeriod, \$endPeriod);
    - foreach (\$period as \$date) { \$date is a datetime object }

# Web Programming

- Server-Side Scripting (PHP) – Mathematical Functions:
  - abs (Absolute value), acos (Arc cosine), acosh (Inverse hyperbolic cosine), asin (Arc sine), asinh (Inverse hyperbolic sine), atan2 (Arc tangent of two variables), atan (Arc tangent), atanh (Inverse hyperbolic tangent), sin(radian value), cos(M\_PI / 4), tan(radian value)
  - echo **abs(-4.2)**; // 4.2 (double/float), echo **abs(5)**; // 5 (integer)

# Web Programming

- Server-Side Scripting (PHP) – OOP:
  - Creating Objects: `$apple = new Fruit; $apple = new Fruit("apple", "20g");`
  - Accessing Properties and methods:
    - `$object->property_name`
    - `$object->method_name([arg, ... ])`
  - Declaring a class

```
class Fruit
{
    public $weight = '';

    function setWeight($newWeight)
    {
        $this->weight = $newWeight;
    }
}
```



# Web Programming

- Advanced JavaScript and XML (AJAX)
  - AJAX (Asynchronous JavaScript and XML): web development technique for creating interactive web applications;
    - send and receive information in formats: JSON, XML, HTML, and text files.
    - "asynchronous" nature: communicate with the server without having to refresh the page.

# Web Programming

- Advanced JavaScript and XML (AJAX)
  - XMLHttpRequest Object: used to interact with servers; retrieve any type of data, not just XML
    - `var httpRequest = new XMLHttpRequest()`
    - `httpRequest.open('GET', 'http://www.example.org/some.file', true);`
    - `httpRequest.send();`
  - Handling Response from Server: write a callback function to handle response before making a request
    - `httpRequest.onreadystatechange = nameOfTheFunction;`

# Web Programming

- Web Development Frameworks
  - Bootstrap: free front end framework for faster and easier web development; collection for creating responsive websites
  - Node.js: cross platform runtime environment for creating server-side tools and apps in JS

```
const server = http.createServer((req, res) => { ... })

server.listen(port, () => {
  console.log(Server running on port: ${*port*})
})
```

# Web Programming

- Web Development Frameworks
  - Angular.js: component-based development platform built on TypeScript; collection of well-integrated libraries i.e. routing; a suite of developer tools
    - Install: `npm install -g @angular/cli`
    - Create project: `ng new <name> --routing=false --style=css`
    - Build and serve: `ng serve`
  - React.js: front-end JavaScript library for building user interfaces based on UI components
    - Create project: `npx create-react-app <name>`